DEVELOPMENT STRATEGY

Repolyx

Software Development Strategy

A simple modern AI development workflow for building Repolyx properly — like a real software engineer.

Version	Status	Type
1.0	Planning	SDLC Document

Introduction

The goal is:

- Structured development
- Faster building using AI
- Less manual coding
- Proper planning
- Clean workflow

Modern software engineering is increasingly moving toward AI-assisted workflows and intelligent development systems instead of writing everything manually.

Phase 01

Idea & Problem Understanding

First understand:

- What problem the app solves
- Who will use it

- Why people need it

Problem

Developers waste time understanding repositories, debugging issues, and reviewing pull requests.

Solution

An AI assistant that understands GitHub repositories and helps developers work faster.

> **Important**
>
> Do not start coding immediately.
>
> First understand: **"Why am I building this?"**

Phase 02

Requirement Planning

Write only important features. Do NOT add too many features initially.

Core Features

- GitHub login
- Repository connection
- Repository scanning
- AI repository chat
- AI PR review
- AI debug assistant

That is enough.

Modern MVP development focuses on small working systems first instead of giant complex products.

Phase 03

UI Planning

Before coding:

- Collect UI inspiration
- Design app screens
- Create simple flow

Take Inspiration From

GitHub | Cursor | Linear | Vercel

Plan Pages

Login Page | Dashboard | Repository Page | AI Chat Page | Review Page

Tools

Use Figma or AI UI generation tools.

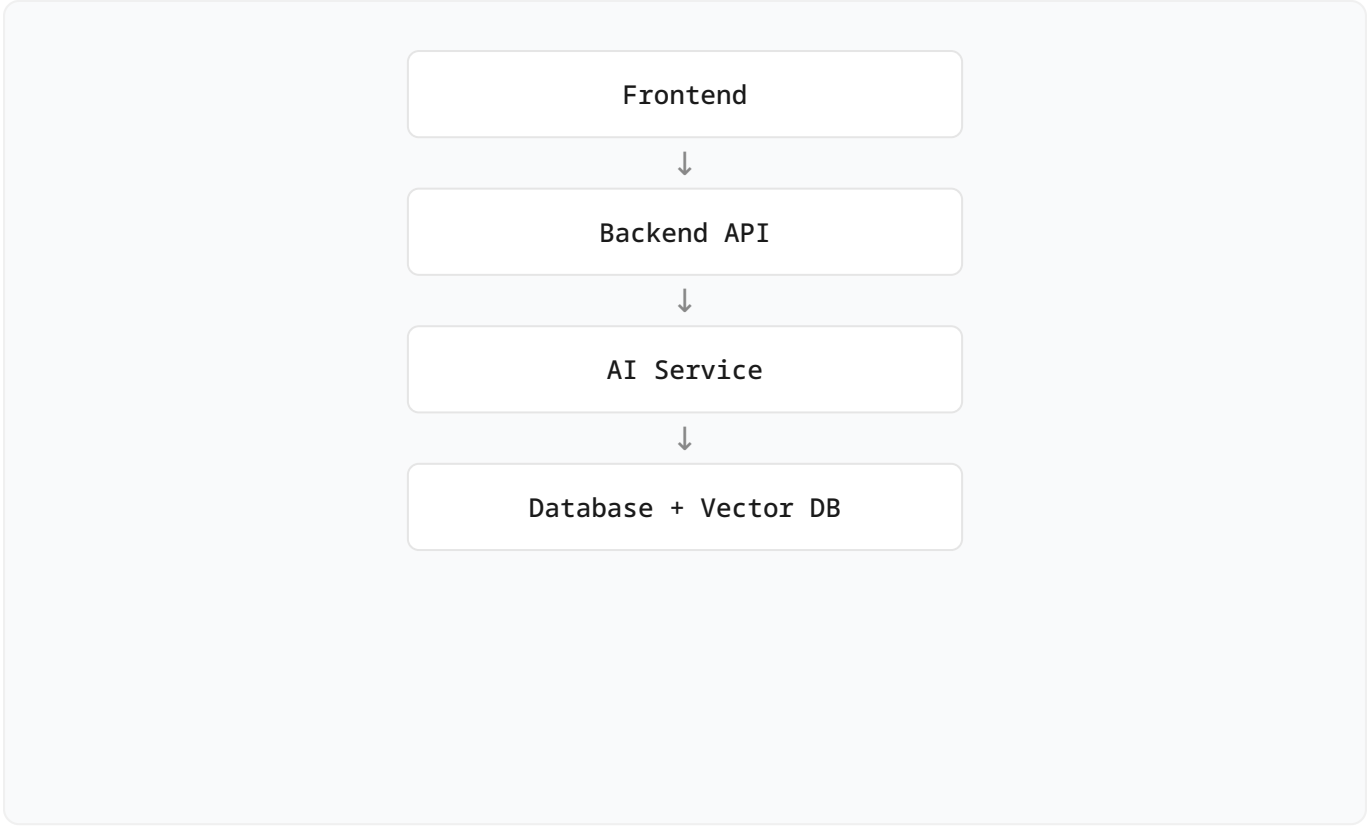
> **Guideline**
>
> Do not waste too much time designing.
>
> Keep UI: **simple, modern, clean.**

Phase 04

Architecture Planning

Plan basic structure before coding.

SIMPLE ARCHITECTURE



This helps avoid confusion later.

Phase 05

Choose Tech Stack

Use modern AI-friendly stack.

FRONTEND	BACKEND	AI
Next.js	Node.js	OpenRouter
Tailwind CSS	Express.js	Claude
ShadCN UI		Gemini
		GPT

DATABASE	VECTOR DATABASE	GITHUB INTEGRATION
PostgreSQL	Qdrant	Octokit API

Guideline

Keep stack simple. Do NOT use too many technologies.

Phase 06

Build Small Modules One by One

Do NOT build full app together. Build feature-by-feature.

Best Order

PHASE 1

Foundation

- GitHub login
- Dashboard
- Repository connection

PHASE 2

Intelligence

- Repository scanning
- Project summary generation

PHASE 3

Chat

- AI repository chat

PHASE 4

Advanced

- PR review
- Debugging assistant

This avoids project overwhelm.

Phase 07

Use AI Properly During Development

Do NOT manually write everything. Use AI as engineering assistant.

Example Workflow

ChatGPT

Use for:

- Architecture planning
- Debugging
- Logic explanation

Claude

Use for:

- Backend coding
- Code reasoning
- API logic

Gemini

Use for:

- Research
- Long-context analysis
- Repository understanding

Modern developers increasingly work with multiple AI systems together instead of one single model.

Phase 08

Build Working Features Fast

Focus on:

- Working product
- Clean workflow
- Proper UX

Do NOT:

- Over-optimize
- Over-engineer
- Build advanced AI agents early

> **Guideline**
>
> A simple working MVP is better than a huge unfinished system.

Phase 09

Testing

After each feature:

- Test immediately
- Fix bugs early

Test

- Login
- Repository fetching
- AI responses
- API routes
- UI flows

Tools

- Postman
- Browser testing
- GitHub test repos

Phase 10

Deployment

Deploy early. Do not wait until project completion.

FRONTEND	BACKEND
Vercel	Railway or Render

DATABASE	VECTOR DB
Neon PostgreSQL	Qdrant Cloud

This makes the project feel real and production-ready.

Phase 11

Documentation

Write:

- Project overview
- Architecture
- Setup steps
- Features
- Screenshots

Good documentation makes projects look professional.

Phase 12

Final Improvement Phase

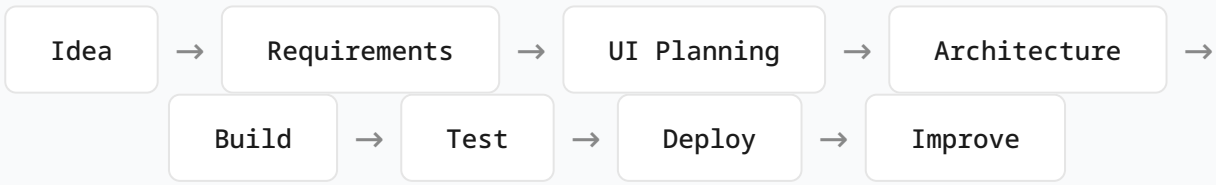
After MVP works:

- Improve UI
- Improve prompts
- Improve AI accuracy
- Optimize workflows
- Polish dashboard

Guideline

Only polish after core features work properly.

Simple Real-World Workflow



Most Important Advice

Modern AI development is NOT:

> "Write everything manually."

Modern AI engineering is:

- Planning properly
- Using AI intelligently

- Building workflows faster
- Focusing on architecture
- Solving real problems

That is exactly what companies increasingly want from developers.